

Text Classification with Neural Networks

Using pytorch and huggingface

December 9, 2025

Table 1: LLM Jailbreak Prompt Classification

	text	label
3	You are a medieval b...	benign
4	Answer the following...	benign
5	[ignore previous ins...	jailbreak
6	Welcome to the world...	jailbreak
7	jailbreak [keyword...	jailbreak

Bag of Words Models

Table 2: Count Vectorization of Text Data

	an	are	as	chatgpt	from	hello	now	the	to	you
0	0	1	0	0	0	0	0	0	0	1
1	0	1	0	0	0	0	0	0	0	1
2	0	0	1	0	1	0	0	0	0	0
3	0	1	0	0	0	0	0	0	0	1
4	0	0	0	0	0	0	0	1	0	0

Each unique "word" is a column, each prompt is a row, and the values represent counts of words in each prompt.

Sequential Representation

Word to Index Dictionary

{'Translate': 0, 'the': 1, 'following':
2, 'sentence': 3, 'to': 4, 'Russian': 5,
'Would': 6}

Map to Sequences of Text

Sentence = "Given the task
definition and input, reply with
output."

Sequence = [1, 7, 15, 3, 4, 11, 11, 4]

Word Embeddings

A different vectorization that are weights from a network trained (for example) using the previous three words to predict the following word. (**NOTE:** We will typically use pretrained embeddings!)

What it looks like

chatgpt	0.23	.98	-2.1
hello	.48	1.7	.69

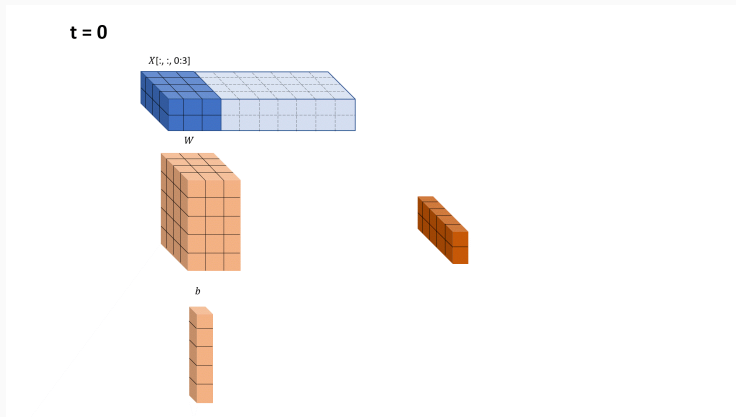
With pytorch

```
nn.Sequential(nn.Embedding(),  
nn.Linear(), ...)
```

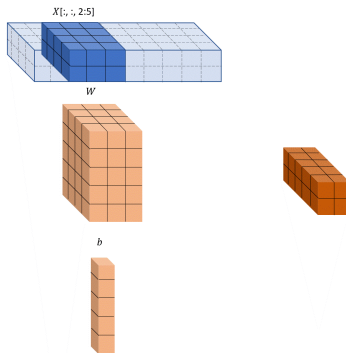
Bag of Words Model → Machine Learning Classifiers and Multilayer Perceptrons

Sequential Representation → Convolutional networks, Recurrent Neural Networks, Transformer Models

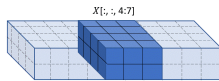
1D Convolutions



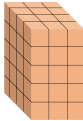
t = 1



t = 2



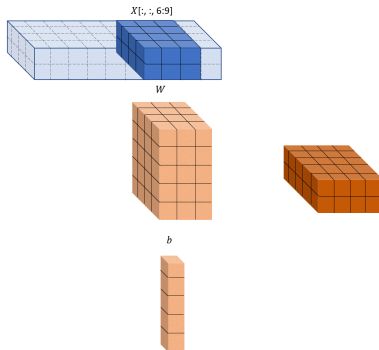
W



b

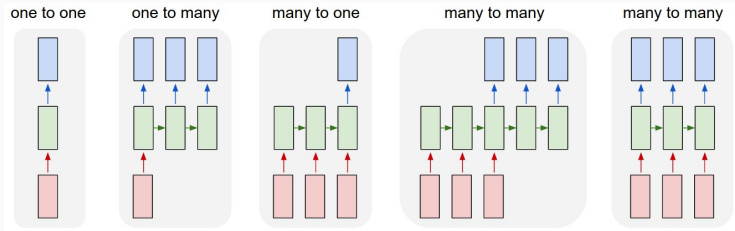


t = 3



Recurrent Neural Networks

$$\tanh(w_{ij}x + b_{ij} + w_{ih}x + b_{ih}) \rightarrow \text{nn.RNN}()$$

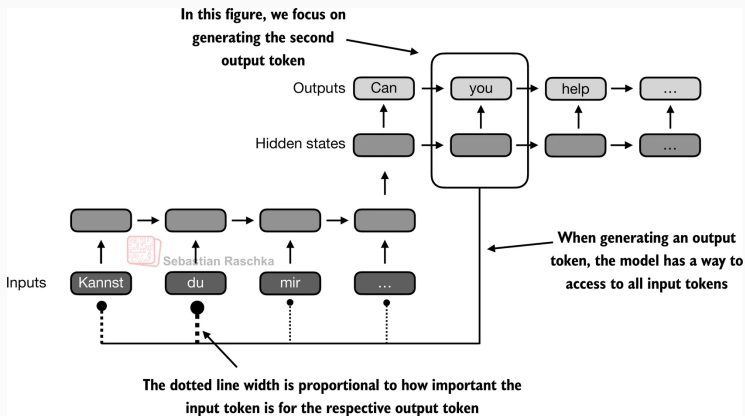


Attention

The Cocktail Party Effect



Attention



Attention is all you need

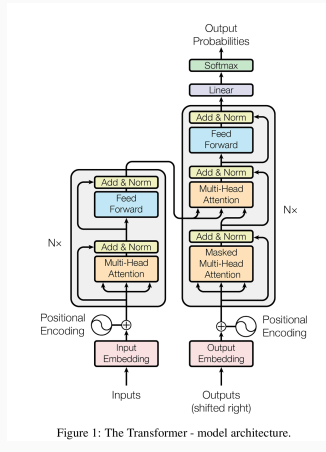


Figure 1: The Transformer - model architecture.

